

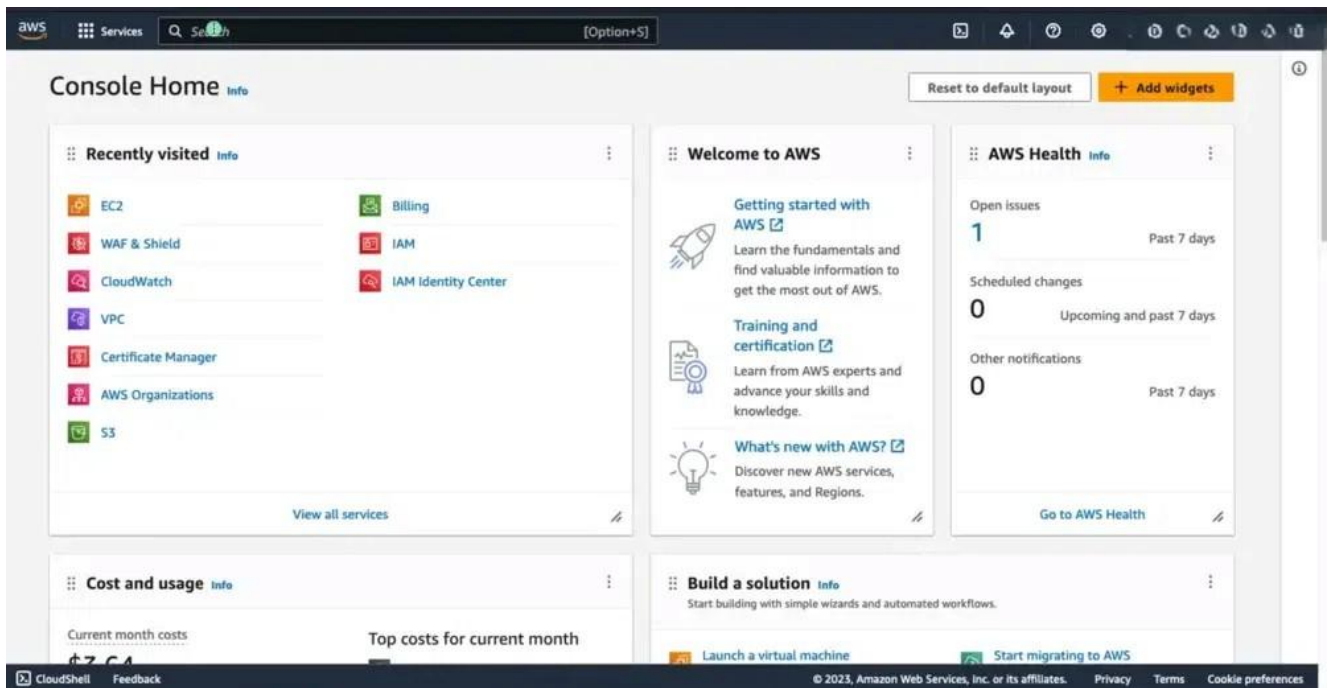
How to Setup AWS VPC Peering Connections: A Step-By-Step Guide

Step 1: Navigate to VPC Dashboard

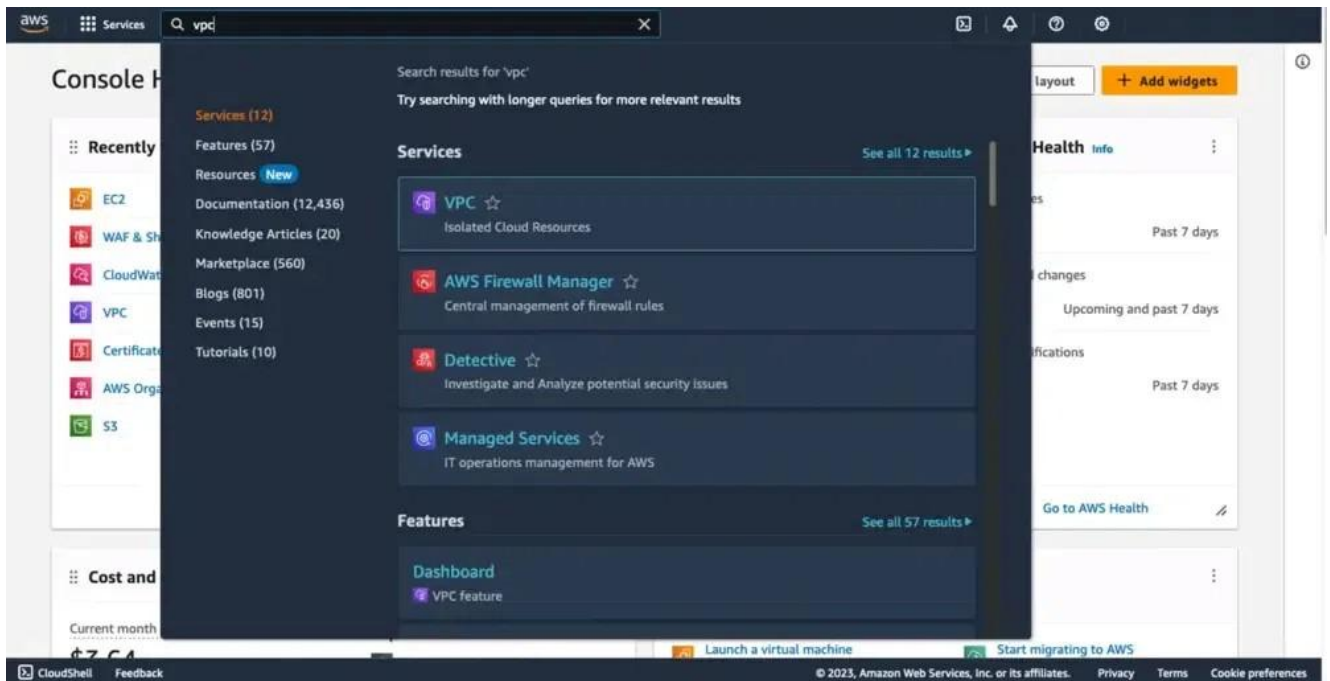
- Sign in to your [AWS account](#) with your credentials.



- After Navigate to Console, you look into the following Console Home Page

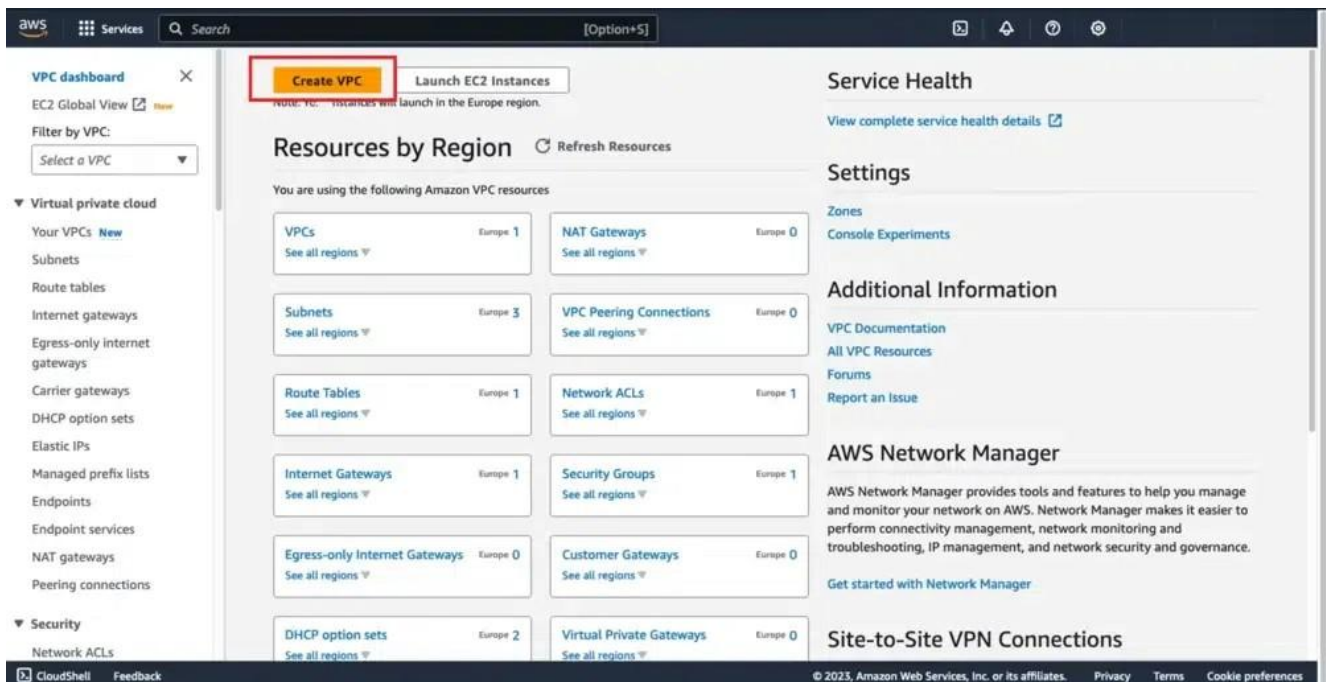


- Select the VPC service in the search box.

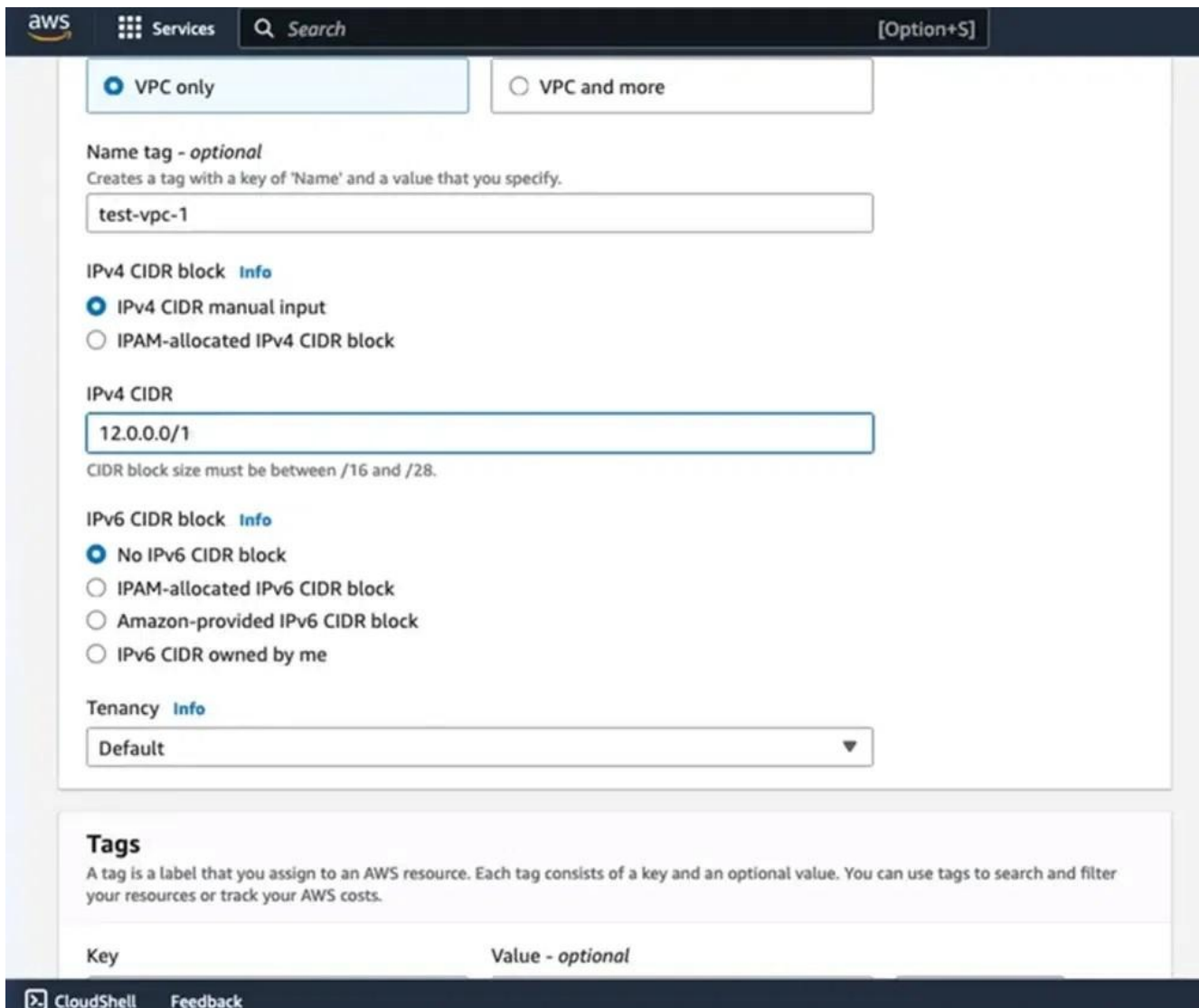


Step 2: Create Virtual Private Network

- Create 2 VPC networks with names test-vpc-1 and test-vpc-2.
- Firstly, Click on the Create VPC button to create VPC.



- Configure the VPC details as shown in the below screenshot.



aws Services Search [Option+S]

☒ IPv4 CIDR manual input
☐ IPAM-allocated IPv4 CIDR block

IPv4 CIDR

12.0.0.0/16

CIDR block size must be between /16 and /28.

IPv6 CIDR block [Info](#)

☒ No IPv6 CIDR block
☐ IPAM-allocated IPv6 CIDR block
☐ Amazon-provided IPv6 CIDR block
☐ IPv6 CIDR owned by me

Tenancy [Info](#)

Default

Tags

A tag is a label that you assign to an AWS resource. Each tag consists of a key and an optional value. You can use tags to search and filter your resources or track your AWS costs.

Key	Value - optional	
Name	test-vpc-1	Remove tag

Add tag

You can add 49 more tags

Cancel Create VPC

- Similarly define and create second VPC with name test-vpc-2 . Ensure that the CIDR of test-vpc-1 and test-vpc-2 doesn't overlay.

aws Services Search [Option+S]

Resources to create [Info](#)

Create only the VPC resource or the VPC and other networking resources.

☒ VPC only
☐ VPC and more

Name tag - optional

Creates a tag with a key of 'Name' and a value that you specify.

test-vpc-2

IPv4 CIDR block [Info](#)

☒ IPv4 CIDR manual input
☐ IPAM-allocated IPv4 CIDR block

IPv4 CIDR

13.0.0.0/16

CIDR block size must be between /16 and /28.

IPv6 CIDR block [Info](#)

☒ No IPv6 CIDR block
☐ IPAM-allocated IPv6 CIDR block
☐ Amazon-provided IPv6 CIDR block
☐ IPv6 CIDR owned by me

Tenancy [Info](#)

Default

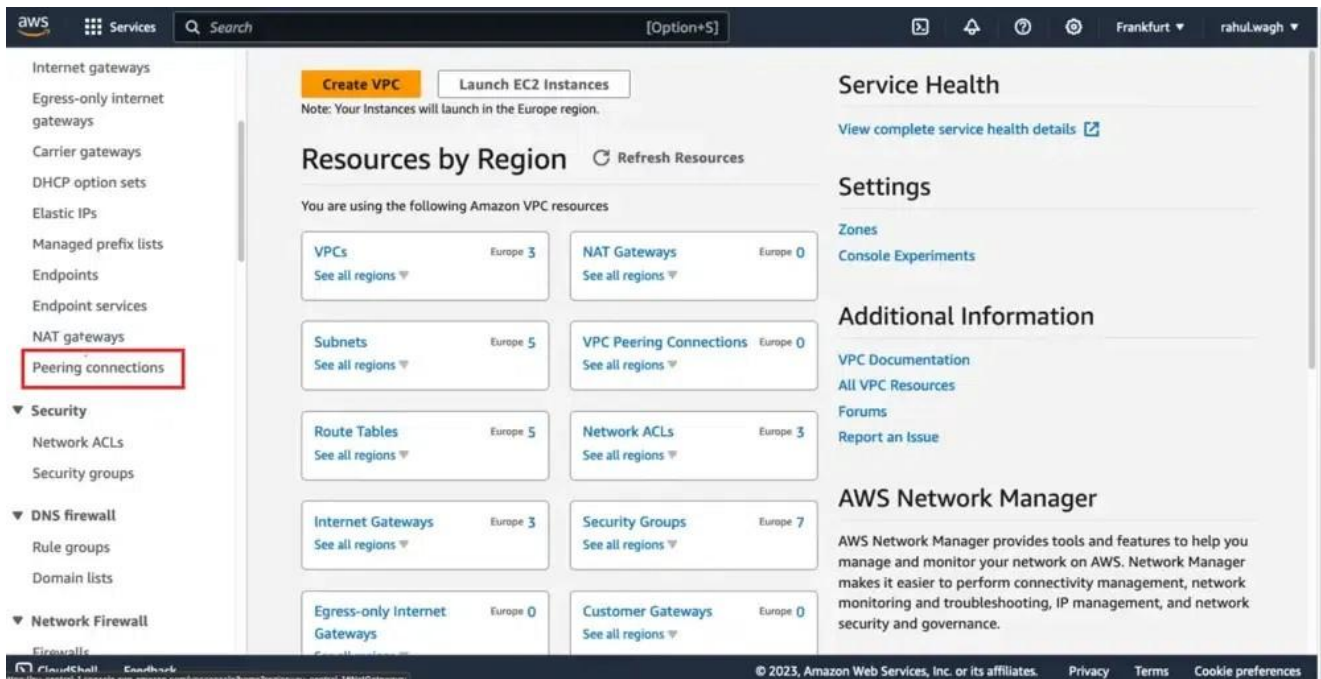
Tags

A tag is a label that you assign to an AWS resource. Each tag consists of a key and an optional value. You can use tags to search and filter your resources or track your AWS costs.

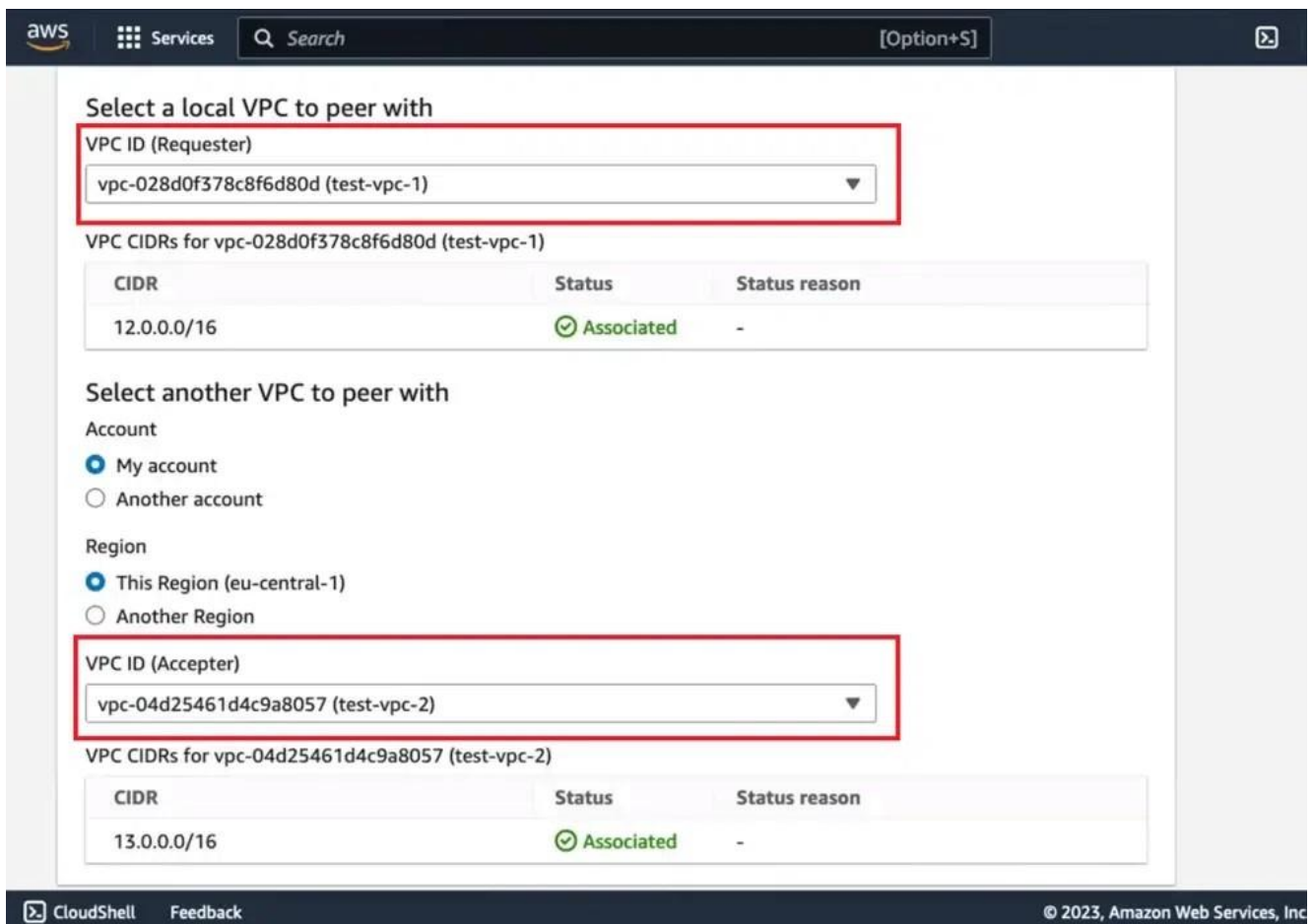
CloudShell Feedback © 2023, Amazon Web Services, Inc.

Step 3: Select "Peering Connections"

- Navigate to VPC Peering, and click on create VPC peering button.

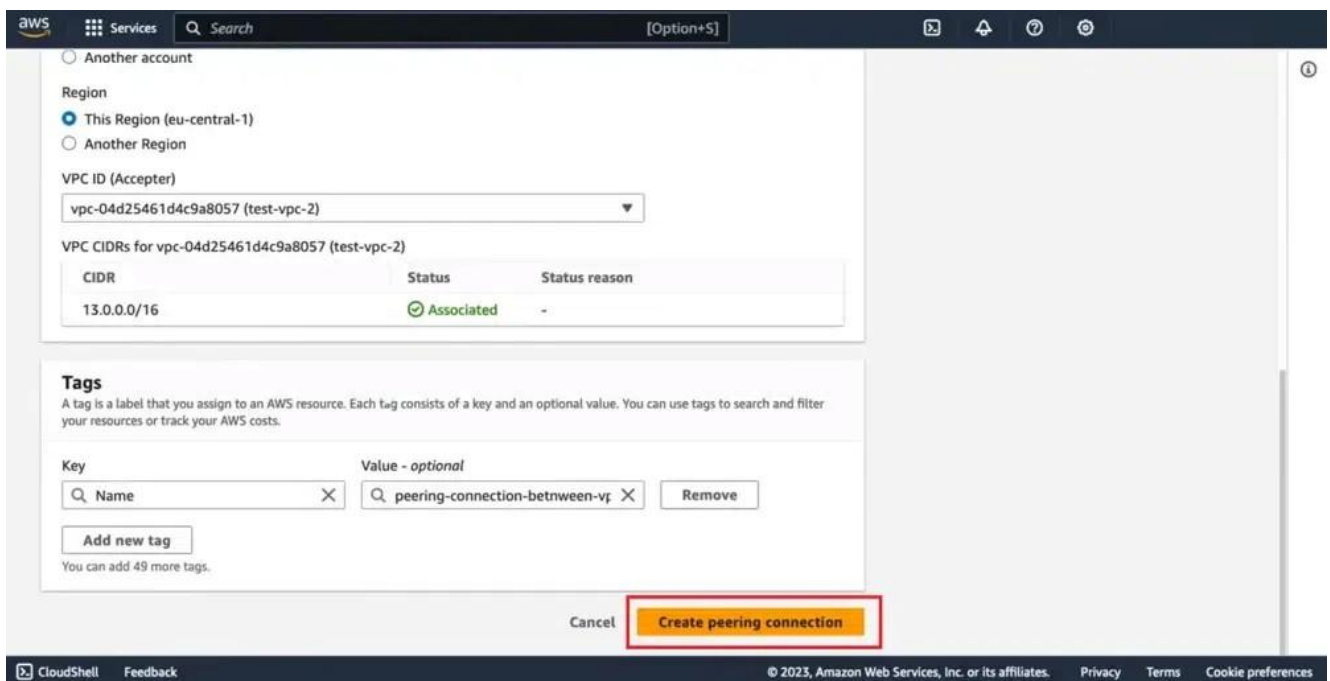


- Provide the acceptor VPC name and receiver VPC name.



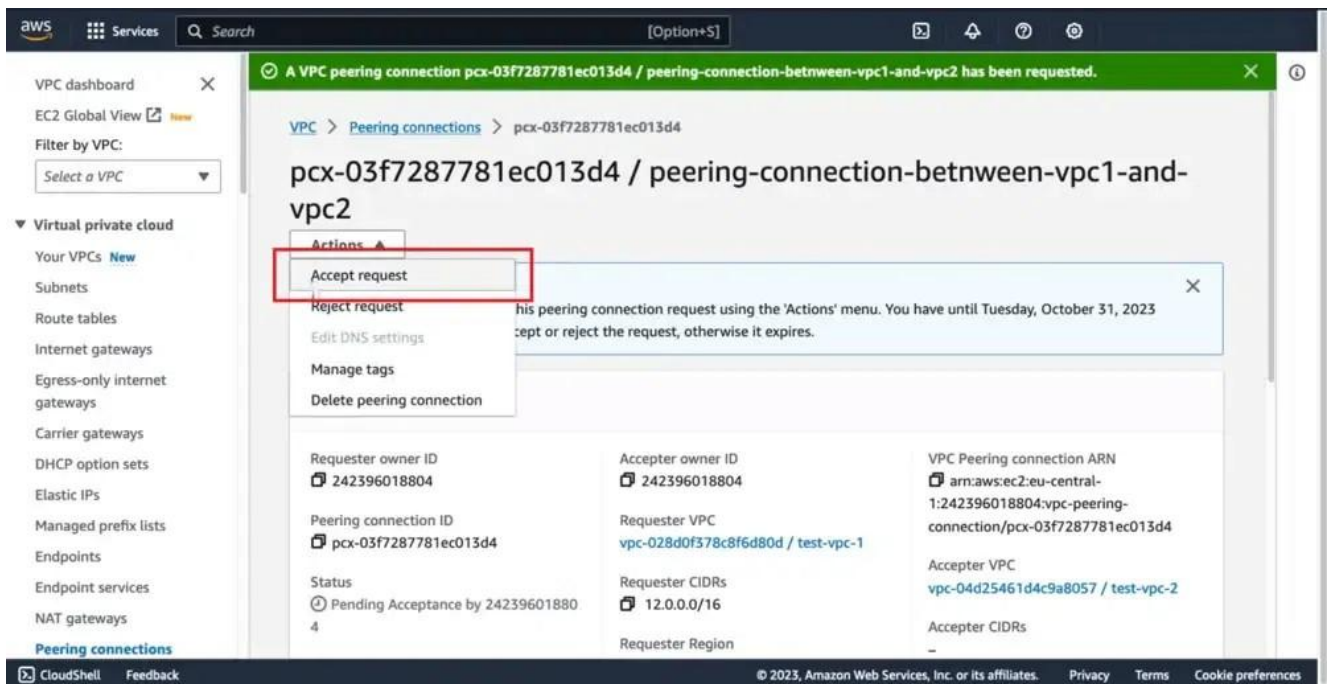
Step 3: Click "Create Peering Connection"

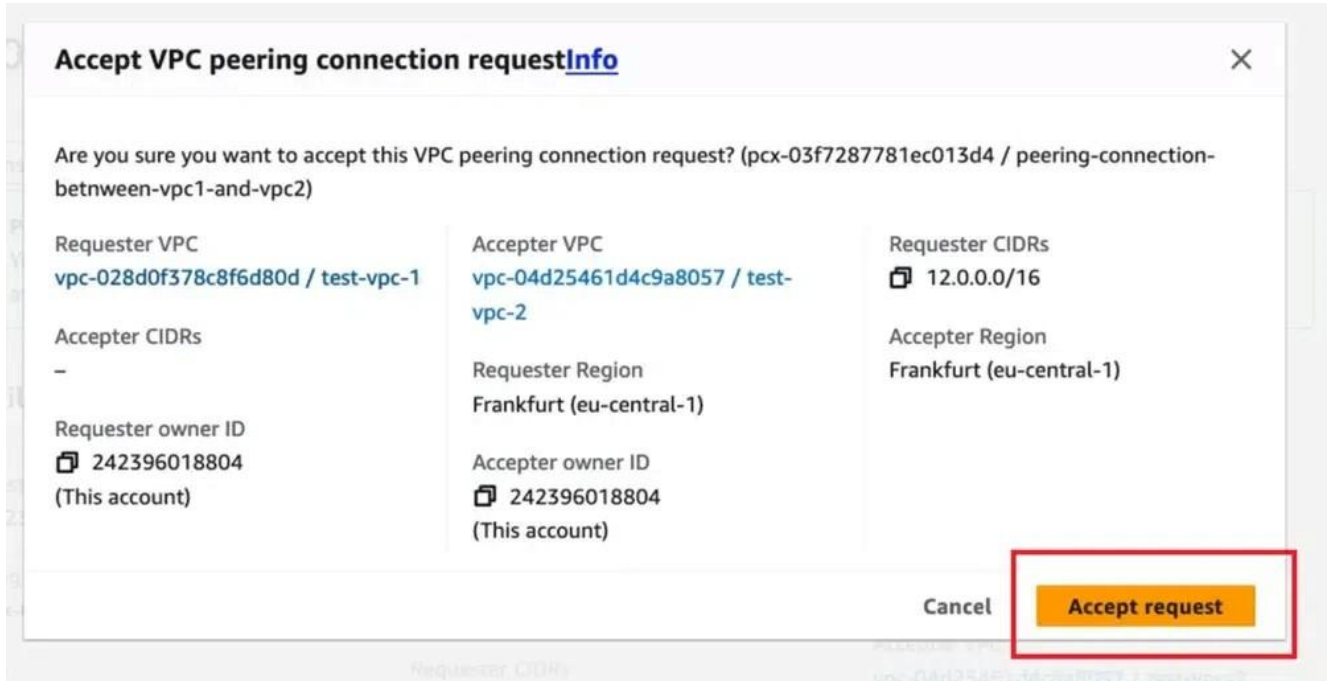
- After creating the VPC peering connection, it will be in pending status .



Step 4: Accept Peering Connection

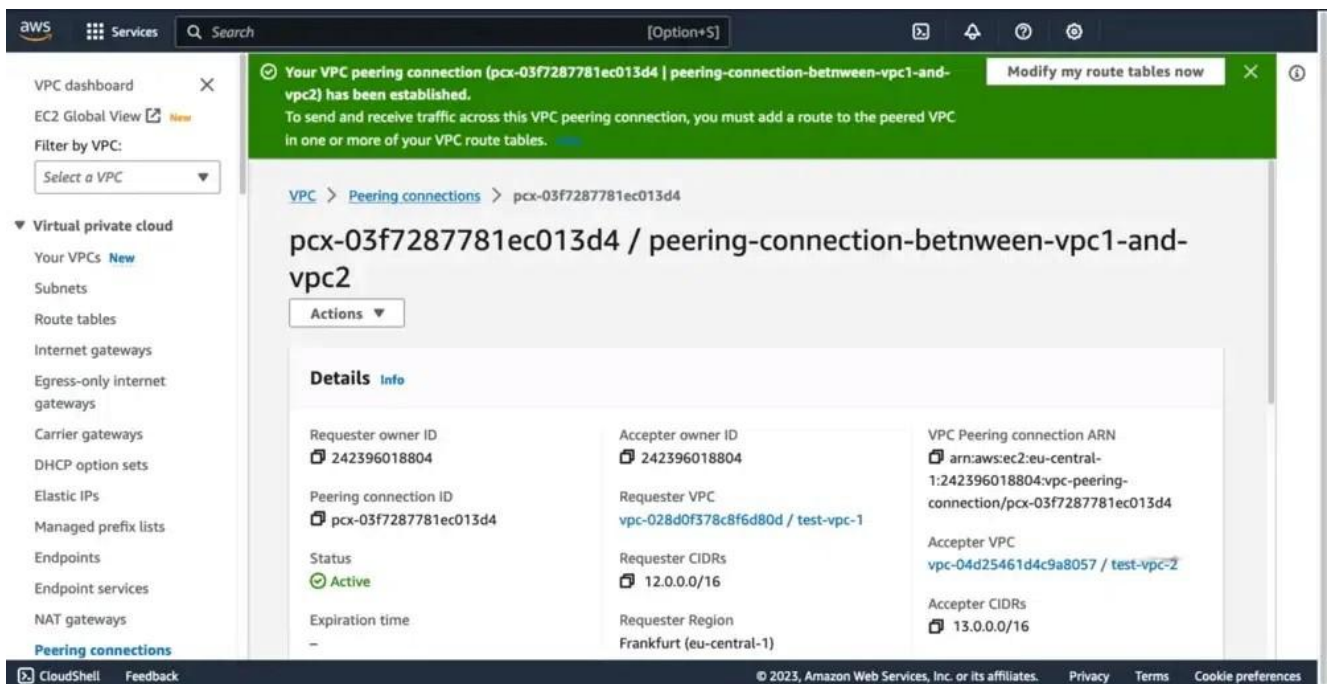
- Now, from the myvpc2 AWS account, go to vpc peering dashboard, click on the **accept request** and accept the peering connection.





Step 5: Successful VPC Peering Connection

- After once the myvpc2 network accepts the request, the VPC peering connection will become active in status.



Step 6: Update Route Tables (if necessary)

- Ensure that the route tables for both VPCs are updated to include routes for each other's CIDR blocks, pointing to the peering connection.

aws Services Search [Option+S]

VPC > Route tables > rtb-0692f1fa6643be455 > Edit routes

Edit routes

Destination	Target	Status	Propagated
12.0.0.0/16	local Q local X	Active	No
Q 0.0.0.0/0 X	Internet Gateway Q igw-0e4c93f13ae7ec926 X	Active	No Remove
Q 13.0.0.0/16 X	Peering Connection Q pcx-03f7287781ec013d4 X	-	No Remove

Add route

Cancel Preview Save changes

aws Services Search [Option+S]

VPC > Route tables > rtb-092b91129746db76a > Edit routes

Edit routes

Destination	Target	Status	Propagated
13.0.0.0/16	local Q local X	Active	No
Q 0.0.0.0/0 X	Internet Gateway Q igw-0cb83609dfaac11de X	Active	No Remove
Q 12.0.0.0/16 X	Peering Connection Q pcx-03f7287781ec013d4 X	-	No Remove

Add route

Cancel Preview Save changes

Step 7: Verify Connectivity

- Now, Create an instance on each VPC and try on ping the private IP of instance on each other instance.
- If they are successfully ping then you have established VPC peering successfully.

```
ubuntu@ip-12-0-1-44:~$ curl 13.0.1.154
<h1>Server Details</h1><p><strong>Hostname:</strong> ip-13-0-1-154</p><p><strong>IP Address:</strong> 13.0.1.154</p>
ubuntu@ip-12-0-1-44:~$

ubuntu@ip-13-0-1-154:~$ curl 12.0.1.44
<h1>Server Details</h1><p><strong>Hostname:</strong> ip-12-0-1-44</p><p><strong>IP Address:</strong> 12.0.1.44</p>
ubuntu@ip-13-0-1-154:~$
```

Sometimes we need to allow resources in different VPCs to exchange data based upon the architecture of the service. Regardless of any specific need, using VPC peering is a great way of extending your VPC design.